# AegisAgent

Yuriko Takahashi

June 23, 2026

**Abstract**

AegisAgent is a negotiation agent developed for the ANAC 2026 Human-Agent Negotiation League (HAN). It combines utility-guarded offer selection, heuristic opponent modeling, time-dependent candidate scoring, and fact-grounded human-aware message generation. The strategic decisions are made by deterministic heuristics based on the utility function and negotiation history. The LLM is used only to phrase human-facing messages after the negotiation action has already been fixed, so the displayed offer and its explanation remain consistent.

## 1 Introduction

AegisAgent is a negotiation agent developed for the ANAC 2026 Human-Agent Negotiation League (HAN). Its goal is to keep strong self-utility, improve the chance of agreement, and produce messages that sound natural to human partners while remaining faithful to the displayed offer. Since HAN evaluates both numerical performance and human-perceived plausibility, the agent is designed to balance strategic strength with message consistency.

The agent does not delegate offer selection, acceptance decisions, candidate generation, utility calculation, or opponent modeling to an LLM. These functions are handled by heuristic scoring over the utility function and negotiation history. The LLM only helps phrase short human-facing responses. This separation keeps the strategy reproducible and efficient while making the response easier to read.

## 2 Overall Architecture

AegisAgent consists of an Offer Decision Module and a Human-Aware Message Generation Module. The Offer Decision Module decides whether to accept the opponent's latest offer or make a counteroffer. It uses the agent's utility function, reservation value, opponent offer and message, offer history, relative time, and scenario type. The Human-Aware Message Generation Module then produces a faithful, human-readable explanation for the negotiation action that has already been fixed.

The processing flow is as follows: (1) obtain the opponent's offer, utterance, time, and history from the negotiation state; (2) estimate opponent type, issue importance, and stubbornness through Opponent Modeling; (3) evaluate candidate outcomes using the Utility-Guarded Core; (4) choose acceptance or a counteroffer; and (5) generate a natural-language message with the Human-Aware Message Generation Module. Figure 1 shows the overall architecture.

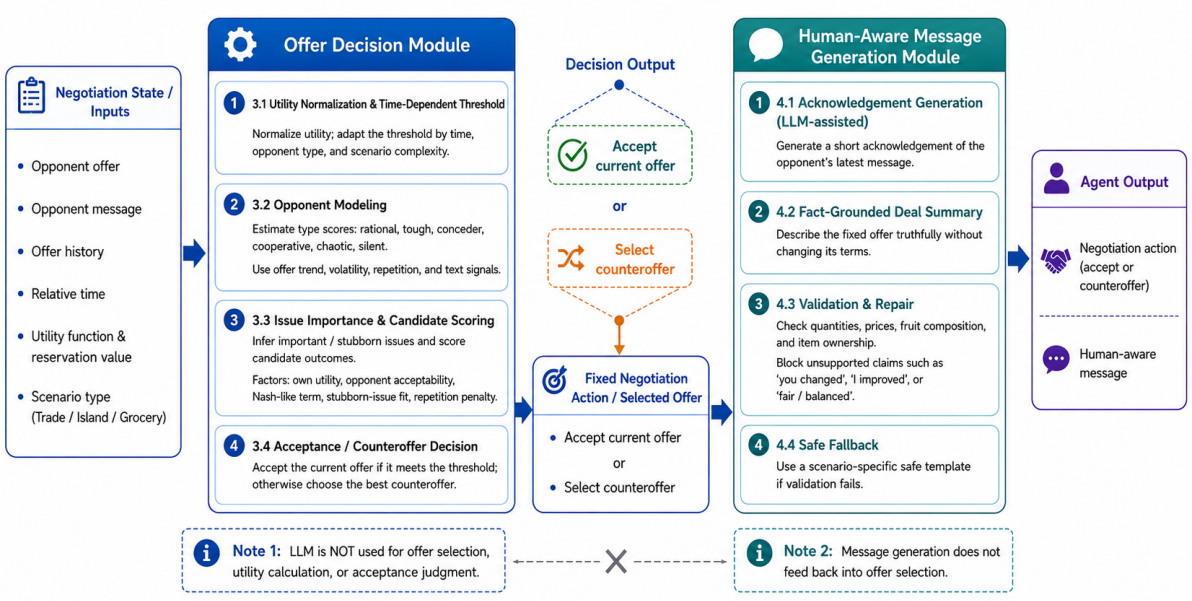



Figure 1: Overall architecture of AegisAgent. The Offer Decision Module determines the negotiation action, and the Human-Aware Message Generation Module produces a faithful explanation of that action.

# 3 Offer Selection and Opponent Modeling

## 3.1 Utility Normalization and Time-Dependent Threshold

To compare utilities across scenarios, the utility $U(o)$ of each outcome $o$ is normalized using the reservation value $U_{\text{res}}$ and the maximum utility $U_{\max}$.

$$\hat{U}(o) = \text{clip}\left(\frac{U(o) - U_{\text{res}}}{U_{\max} - U_{\text{res}}}, 0, 1\right) \tag{1}$$

Let $t \in [0, 1]$ denote relative negotiation time, and let $A(t)$ denote the base aspiration level.

$$A(t) = A_{\text{end}} + (A_{\text{start}} - A_{\text{end}})(1 - t^p) \tag{2}$$

A correction term $\Delta(t)$ based on opponent type and scenario complexity is then added. The final threshold for acceptance and proposal is defined as

$$\theta(t) = \text{clip}(A(t) + \Delta(t), F(t), 0.995) \tag{3}$$

where $F(t)$ is a utility floor. It is high early in the negotiation and decreases later to reduce the risk of no agreement. Against tough or chaotic opponents, the threshold protects self-utility more strongly. Against rational, cooperative, or conceding opponents, it is relaxed to increase the chance of agreement.

## 3.2 Estimation of Opponent Types

In Opponent Modeling, the agent does not try to learn the opponent's true utility function directly. Instead, it estimates behavioral tendencies from the observed offer and utterance histories.

Table 1: Opponent types used in Opponent Modeling

| Type | Description | Main cues |
|---|---|---|
| Rational | Gradually makes reasonable concessions and moves toward agreement. | High average utility $A$ and positive utility trend $T > 0$. |
| Tough | Keeps demanding terms and often repeats similar offers. | Low concession and high repetition ratio $R$. |
| Conceder | Makes relatively large concessions over time. | Large positive trend $T$ and decreasing utility in later rounds. |
| Cooperative | Shows a cooperative attitude both in language and proposals. | Positive text signal $P$ and continued offer improvement. |
| Chaotic | Produces highly variable offers with weak consistency. | High volatility $V$ and frequent abrupt changes. |
| Silent | Provides little verbal evidence. | Very few messages and limited text signals. |

For each turn, the opponent offer $o_k$, relative time $t_k$, normalized self-utility $\hat{U}(o_k)$ of that offer, and opponent message $m_k$ are stored in the history $H$:

$$H = \{(o_k, t_k, \hat{U}(o_k), m_k)\}_{k=1}^{n} \tag{4}$$

From this history, the agent computes average utility $A$, latest utility $L$, utility trend $T$, volatility $V$, repeated-offer ratio $R$, positive text signal $P$, and negative text signal $N$. Textual features are used only as supporting signals; actual movements in the opponent's offers are given higher priority. The opponent type is represented by six continuous scores: rational, tough, conceder, cooperative, chaotic, and silent. The coefficients in this section were not learned. They were empirically set to map offer utility, concession tendency, repetitiveness, volatility, and utterance features to negotiation types. Larger weights were assigned to concession and volatility signals, which depend on small changes, while utterance features remained auxiliary.

Rational opponents tend to maintain relatively high utility with a positive trend. Tough opponents keep demanding offers and repeat similar proposals. Conceders show clearer concessions over time. Cooperative opponents combine positive language with proposal improvements. Chaotic opponents make volatile offers, and silent opponents provide limited linguistic evidence.

### 3.3 Issue Importance and Candidate Scoring

AegisAgent also estimates which issues the opponent seems to care about most. For issue $i$, importance is computed from repetition degree $r_i$, change rate $c_i$, and mention bonus $b_i$ as

$$q_i = \max(0.05, 0.55r_i + 0.35(1 - c_i) + b_i), \quad I_i = \frac{q_i}{\sum_j q_j} \tag{5}$$

Issues that are often repeated, rarely changed, or explicitly mentioned are treated as more important. The system also estimates issue-wise flexibility from the change rate and observed range, and treats low-flexibility issues as stubborn issues.

Candidate offer $o$ is evaluated by combining basic opponent fit and priority-weighted fit. The final score also includes self-utility, opponent acceptability, a Nash-like product term, fit

to stubborn issues, a patience bonus against conceders, and a repetition penalty:

$$\text{Score}(o,t) = \text{Own}(o,t) + \text{Opp}(o,t) + \text{Nash}(o,t) + \text{Stubborn}(o,t) + \text{Bonus} - \text{Penalty} \quad (6)$$

Here, $\text{Own}(o,t)$ is the agent's normalized utility, weighted by time and opponent type so that utility is protected more strongly early or against tough and chaotic opponents. $\text{Opp}(o,t)$ estimates acceptability from similarity to the opponent's past offers and important issues. $\text{Nash}(o,t)$ rewards candidates that are strong for the agent and likely acceptable to the opponent, reducing one-sided choices. $\text{Stubborn}(o,t)$ checks whether the offer respects issues on which the opponent has rarely moved. Bonus delays over-adaptation to conceders, and Penalty discourages repeated offers that stall the dialogue. In this way, Equation (6) balances own utility, predicted acceptability, important issues, and avoidance of unproductive repetition.

The selected counteroffer is the outcome that maximizes this score in the candidate set $C_t$,

$$C_t = \{o \mid U(o) \geq \theta(t)\} \quad (7)$$

and if no candidate satisfies the threshold, fallback candidates are constructed from high-utility outcomes. In Trade, evaluation focuses on Quantity and Price; in Island, on item ownership; and in Grocery, on fruit composition and variety balance.

### 3.4 Acceptance Strategy

For acceptance, the agent computes the normalized self-utility of the opponent's latest offer $o_{\text{opp}}$ and generally accepts when

$$\hat{U}(o_{\text{opp}}) \geq \theta(t) \quad (8)$$

Near-threshold offers may also be accepted when the opponent is likely rational, cooperative, or a conceder. In the final stage, acceptance is relaxed to reduce no-agreement risk, while extremely low-utility agreements are still avoided.

In the final implementation, this late-stage behavior is handled by an additional acceptance and rescue layer. When the deadline is close, the agent compares the opponent's latest offer against relaxed self-utility floors and may accept offers that fall below the normal threshold but remain above a minimum safety level. If agreement appears unlikely, the agent may also choose a rescue counteroffer that gives greater weight to predicted acceptability and the opponent's stubborn issues, while still avoiding clearly harmful low-utility agreements.

## 4 Human-Aware Message Generation

This section summarizes the overall flow of the Human-Aware Message Generation Module shown on the right-hand side of Figure 1. The module receives the negotiation action already fixed by the Offer Decision Module, either an acceptance decision or a selected counteroffer, and generates a short explanation for the human partner. Message generation never feeds back into offer selection: the LLM may help with wording, but it does not change the numerical offer, item ownership, fruit composition, or acceptance decision. In this implementation, the LLM model was qwen3:4b-instruct.

The flow is to extract the issue raised in the opponent's message, structure the facts of the fixed offer, generate a short response with the LLM, and finalize the output through validation and repair or safe fallback. The following subsections describe Acknowledgement Generation, Fact-Grounded Deal Summary, Validation and Repair, and Safe Fallback.

Table 2: Text features used in Acknowledgement Generation

| Feature | Meaning |
|---|---|
| mentioned_items | Items, fruits, or trade terms explicitly named in the opponent's message. |
| primary_mentioned_item | The main item or issue to acknowledge when several terms are mentioned. |
| explicit_trade_pair | Whether the Trade message explicitly refers to both Quantity and Price. |
| expresses_split_concern | Whether the opponent expresses concern about the split, mix, variety, or ownership of items. |
| asks_for_reason | Whether the opponent asks for an explanation or justification of the offer. |

### 4.1 Acknowledgement Generation

A short acknowledgement is generated for the opponent's utterance. This is not a generic statement such as "I understand." Instead, it responds only to the issue the opponent actually raised. In Trade, when Quantity and Price are both stated, both are treated as known issues, and the agent does not ask the user to choose which matters. In Grocery, remarks about variety, fruit mix, or named fruits are prioritized. In Island, utterances about item ownership, such as hammer, rope, or food, provide the main context.

In the implementation, a text-feature extraction step analyzes the opponent's text and creates the features summarized in Table 2.

Based on these features, the reaction-option generation step prepares a small set of possible opening reactions. For example, it can acknowledge both Quantity and Price in Trade, respond to a variety concern in Grocery, or refer to a named item in Island. The selected reaction must match the extracted facts, so the agent can respond directly to known issues without guessing hidden preferences.

### 4.2 Fact-Grounded Deal Summary

The fixed offer is explained in a fact-grounded way. This stage turns the already selected offer into a human-readable description, rather than creating new negotiation terms. The prompt receives structured information such as the current offer terms, the opponent's offer terms, changes from the previous offer, the scenario family, and the message intent.

In the implementation, a structured fact extraction step determines whether the opponent changed their offer, whether our offer changed, whether the current offer is repeated, the direction of the human-visible utility change, backtrack status, and message intent. These facts control which explanations are allowed. A holding expression is used only when our offer is actually unchanged, while expressions such as "improved" or "step toward you" are avoided unless the human-visible share has increased. In addition, offer-term extraction supports Quantity and Price in Trade, fruit counts in Grocery, and item assignments in Island.

### 4.3 Validation and Repair

The generated message is validated and repaired when possible. Validation checks both numerical consistency and claims about the negotiation flow. The system rejects messages that

say “you changed” when the opponent’s offer did not change, “same package” when our offer is not the same, or “improved” when the human-visible utility did not increase. Unsupported evaluative claims such as “fair,” “balanced,” or “mutually beneficial,” as well as guesses about hidden user preferences, are also blocked.

In the implementation, explanation validation and factuality guards detect false opponent movement, false same-offer claims, false improvement claims, unsupported balance claims, and unsupported user inference. The repair step rewrites messages that ask again about an already known issue, ask about only one side of a Trade request when both Quantity and Price were given, or use stiff language that does not fit the scenario. If the repaired message still fails validation, the LLM output is discarded.

### 4.4 Safe Fallback

LLM timeouts, validation failures, very short remaining time budgets, and prompt-injection-like inputs are handled by safe fallback. The system then returns a short template selected by scenario and message intent. It does not exaggerate offer terms or make unsupported claims about fairness or concession.

## 5 Evaluation and Discussion

The evaluation examined negotiation performance, strategic stability, message quality, human-perceived naturalness, and safety. Grocery, Trade, and Island scenarios covered symmetric and asymmetric preferences and strong conflicts of interest. The agent was compared with tough, conceding, time-dependent, and reciprocal baselines, and regression tests checked that the final revision did not substantially harm numerical performance.

During development, we considered using an LLM as a user-like acceptance judge to evaluate how acceptable candidate offers would look from the human side. This design increased processing time and, under short time limits, caused more timeouts and fallback events. An LLM-based selector also did not always choose the strongest candidate stably, reducing reproducibility.

The coefficients used in this agent were not optimized by learning; they are heuristic weights designed to map offer utility, concession tendency, repetitiveness, volatility, and utterance features to strategic signals.

## 6 Conclusion

AegisAgent is a HAN negotiation agent that combines utility-guarded offer selection, heuristic opponent modeling, time-dependent candidate scoring, and fact-grounded message generation. By separating offer selection from message generation and adding validation and repair to the latter, the agent balances numerical strength with naturalness for human interaction.